

Lesson 6 — K-Nearest Neighbors (KNN)

So far:

- Lesson 1: ML basics
- Lesson 2: Linear Regression
- Lesson 3: Regression Evaluation
- Lesson 4: Classification
- Lesson 5: Classification Evaluation
-  Lesson 6: K-Nearest Neighbors (KNN)

KNN is one of the easiest ML algorithms to understand because its basic idea is very intuitive:

Similar things tend to have similar outcomes.

1. What is KNN?

KNN stands for:

K-Nearest Neighbors

Suppose we have some points belonging to two classes:

```
[B] [B]
[B]
      [R]
    [R] [R]
      [?]
```

We want to classify [?] .

The algorithm looks at the points closest to it.

If most nearby points are [B]:

```
[B] [B]
[B]
      [?]
      [R]
```

Then:

```
[?] → [B]
```

If most nearby points are [R]:

```
[B]
      [?]
    [R] [R]
      [R]
```

Then:

```
[?] → [R]
```

That's the basic idea behind KNN.

2. What Does “K” Mean?

This is very important.

K = number of nearest neighbors we look at.

Suppose:

```
K = 3
```

Then the model looks at the 3 closest data points.

Example:

```
New point
↓
Find 3 nearest points
↓
[B] [B] [R]
↓
Majority = [B]
↓
Prediction = [B]
```

So:

```
K = 3
```

means:

Look at the 3 nearest neighbors.

3. Example With K = 5

Suppose the 5 nearest neighbors are:

```
[B] [B] [B] [R] [R]
```

Count:

```
[B] = 3
[R] = 2
```

Majority:

```
[B]
```

Therefore:

```
Prediction = [B]
```

4. Why Is It Called “Nearest”?

Because KNN calculates the distance between points.

For example:

```
A ----- B
```

If A and B are very close:

```
distance = small
```

If they're far apart:

```
A ----- B  
distance = large
```

KNN uses distance to determine which observations are the nearest.

5. Real-World Example

Imagine you want to classify a fruit as:

```
☒ Apple  
☐ Orange
```

You have two features:

```
Weight  
Color
```

Suppose:

```
Apple:  
Weight = 150g  
Color = Red  
  
Orange:  
Weight = 180g  
Color = Orange
```

Now you receive a new fruit:

```
Weight = 155g  
Color = Red
```

It's probably an apple because its characteristics are similar to apples.

KNN essentially says:

“Let's look at the fruits most similar to this one.”

6. KNN in Scikit-learn

For classification, use:

```
from sklearn.neighbors import KNeighborsClassifier
```

Create the model:

```
model = KNeighborsClassifier(n_neighbors=5)
```

Here:

```
n_neighbors=5
```

means:

```
K = 5
```

Train:

```
model.fit(X_train, y_train)
```

Predict:

```
predictions = model.predict(X_test)
```

7. Complete Example

Let's use the same Breast Cancer dataset from Lesson 4.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()

X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

8. Something Interesting About KNN

You might be wondering:

“Where is the learning?”

With some algorithms, training involves finding mathematical parameters.

KNN is different.

During `.fit()` it essentially stores the training data so it can use it later when making predictions.

When you call:

```
model.predict(X_test)
```

KNN:

```
New data point
↓
Calculate distances
↓
Find nearest K points
↓
Look at their classes
↓
Majority vote
↓
Prediction
```

9. Why K Matters

Let's say:

```
K = 1
```

The model looks at only one neighbor.

```
New point
↓
Closest point
↓
Its class
```

This can be very sensitive to individual data points.

If:

```
K = 5
```

we consider five neighbors.

```
5 neighbors
↓
Majority vote
↓
Prediction
```

Generally, increasing K makes the decision less sensitive to individual observations.

10. K = 1 vs K = 5

Imagine:

```
[B] [B] [B]
      [?]
      [R]
```

If the nearest point happens to be [R]:

```
K = 1
[?] → [R]
```

But with:

```
K = 5
```

the model considers more surrounding points and might find:

```
[B] [B] [B] [R] [R]
```

Therefore:

```
[?] → [B]
```

11. But Should We Always Use a Huge K?

No.

Suppose:

```
K = 1000
```

and your dataset has 1200 observations.

Now the model considers a huge portion of the dataset.

The prediction can become too influenced by the overall majority class rather than the local neighborhood.

So choosing K is important.

Later we'll learn how to choose it systematically using hyperparameter tuning and cross-validation.

12. One BIG Problem With KNN — Feature Scale

This is extremely important.

Suppose our features are:

```
Age      → 20-80
Salary   → 20,000-2,00,000
```

KNN uses distance.

Salary has much larger numerical values than age.

Therefore salary can dominate the distance calculation.

For example:

```
Age difference    = 5
Salary difference = 50,000
```

The salary difference is numerically much larger.

So we usually need feature scaling before using KNN.

13. What is Feature Scaling?

Suppose:

```
Age:
20
30
40
50
```

and:

```
Salary:
20,000
50,000
100,000
200,000
```

We transform the features to comparable scales.

For example:

```
Age      → -1.2, -0.5, 0.3, 1.1
Salary   → -1.0, -0.6, 0.1, 1.5
```

Now distance calculations are much more balanced.

We'll study scaling properly in a later lesson.

14. StandardScaler

Scikit-learn provides:

```
from sklearn.preprocessing import StandardScaler
```

Create:

```
scaler = StandardScaler()
```

Fit and transform training data:

```
X_train_scaled = scaler.fit_transform(X_train)
```

Transform test data:

```
X_test_scaled = scaler.transform(X_test)
```

Notice something important:

Training data

```
scaler.fit_transform(X_train)
```

Test data

```
scaler.transform(X_test)
```

We don't fit the scaler on test data.

Why?

Because the test set should remain unseen during training.

We'll explore this carefully in the preprocessing lesson.

15. KNN With Scaling

Here's a better version:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load data
data = load_breast_cancer()

X = data.data
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# KNN
model = KNeighborsClassifier(n_neighbors=5)

# Train
```



```
model.fit(X_train_scaled, y_train)

# Predict
predictions = model.predict(X_test_scaled)

# Evaluate
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

You'll generally see that scaling makes KNN perform better on datasets where features have different scales.

16. Why Doesn't fit() Look Like It Does Much?

This is a subtle but useful point.

For:

```
LinearRegression()
```

the model learns coefficients.

For:

```
KNeighborsClassifier()
```

the model mainly retains the training examples.

Then during prediction:

```
predict()
```

it asks:

“Which training points are closest to this new point?”

That's why KNN is often called a lazy learner.

17. KNN vs Logistic Regression

You now know two classification algorithms.

Logistic Regression

```
Data
↓
Learn mathematical relationship
↓
Decision
```

KNN

```
Data
↓
Find nearby examples
↓
Majority vote
↓
```

Decision

So:

	Logistic Regression	KNN
Type	Classification	Classification
Main idea	Learn relationship	Find neighbors
Uses distance	✗	✓
Scaling important	Usually helpful	Very important
Prediction	Fast	Can be slower
Easy to understand	✓	✓

18. KNN Regression Also Exists

You may remember from an earlier question:

```
from sklearn.neighbors import KNeighborsRegressor
```

Yes!

KNN can be used for both regression and classification.

Classification

```
KNeighborsClassifier()
```

Predicts:

```
0 / 1
```

Regression

```
KNeighborsRegressor()
```

Predicts:

```
25.4  
72.8  
103.2
```

The difference is what happens with the neighbors.

Classification

Majority vote:

```
[B] [B] [R]  
  ↓  
 [B]
```

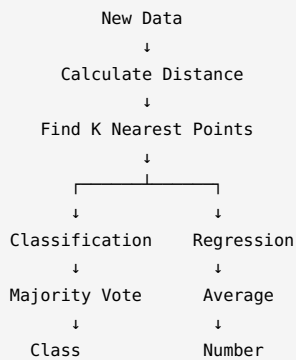
Regression

Average of neighbors:

20
30
40
↓
Average = 30

19. The Most Important KNN Flow

Memorize this:



🧠 What You Should Remember From Lesson 6

KNN means:

K-Nearest Neighbors

K means:

Number of neighbors considered

Classification:

```
KNeighborsClassifier(n_neighbors=5)
```

Training:

```
model.fit(X_train, y_train)
```

Prediction:

```
model.predict(X_test)
```

KNN classification:

Nearest neighbors
↓
Majority vote
↓
Class

KNN regression:

Nearest neighbors

↓
Average
↓
Number

Most important practical point:

Because KNN relies on distance, feature scaling is usually very important.

Exercises

Try these without looking back:

1. What does K mean in KNN?
2. If $K = 3$ and the neighbors are:

0, 1, 1

what will the model predict?

3. If $K = 5$ and neighbors are:

0, 0, 0, 1, 1

what is the prediction?

4. Why is feature scaling important for KNN?
5. What's the difference between:

`KNeighborsClassifier()`

and

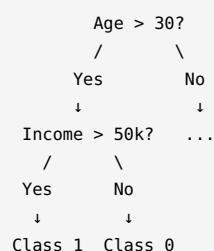
`KNeighborsRegressor()`

6. Why can $K = 1$ be sensitive to individual data points?
7. What does this mean?

`KNeighborsClassifier(n_neighbors=5)`

Next Lesson — Lesson 7: Decision Trees 🌲

We'll learn one of the most useful ML algorithms:



We'll understand how a Decision Tree actually decides where to split the data, then implement `DecisionTreeClassifier` in Scikit-learn.